

Plan de formación de 30 días

Desarrollador C#/.NET con Angular y ORM Entity Framework

Enfoque: Clean Architecture, ASP.NET Core Web API, Angular, Entity Framework Core, seguridad, pruebas, DevOps, Kubernetes y AWS

Proyecto guía: plataforma de gestión de órdenes, productos, inventario, pagos, notificaciones y frontend administrativo.

Documento adaptado a partir del plan original de formación backend de 30 días.

Resultado esperado al finalizar

Al terminar los 30 días, el desarrollador debe poder:

- Diseñar aplicaciones full stack desacopladas usando Clean Architecture o arquitectura hexagonal.
- Implementar APIs REST con ASP.NET Core Web API.
- Aplicar principios SOLID, Clean Code y buenas prácticas en C#.
- Construir interfaces web con Angular usando componentes, servicios, formularios y rutas.
- Implementar persistencia usando Entity Framework Core.
- Diseñar entidades, relaciones, migraciones, repositorios y consultas eficientes.
- Implementar validaciones, autenticación, autorización y manejo seguro de errores.
- Crear pruebas unitarias, pruebas de integración y pruebas frontend.
- Documentar APIs usando OpenAPI/Swagger.
- Construir imágenes Docker listas para ambientes reales.
- Crear pipelines CI/CD.
- Desplegar aplicaciones en Kubernetes o servicios cloud.
- Aplicar buenas prácticas de seguridad, observabilidad y mantenibilidad.

Supuestos de trabajo

- Duración: 30 días calendario.
- Perfil inicial: desarrollador middle con experiencia básica en backend o frontend.
- Modalidad: teoría + práctica + seguimiento semanal.
- Ejercicios prácticos: cada 5 días.
- Metodología: Scrum, usando historias de usuario, criterios de aceptación, refinamiento, revisión y retrospectiva.
- Proyecto guía: plataforma de gestión de órdenes, productos, inventario, pagos y notificaciones.

Proyecto guía del mes

Contexto funcional

Se construirá una plataforma modular para gestionar órdenes de compra. El sistema debe permitir:

- Crear órdenes.
- Consultar órdenes.
- Gestionar productos.
- Validar inventario.
- Calcular totales.
- Procesar pagos simulados.
- Enviar notificaciones.
- Consultar el estado de una orden.
- Administrar usuarios y roles.
- Visualizar órdenes desde una aplicación Angular.

Módulos funcionales sugeridos

- Gestión de órdenes.

- Catálogo de productos.
- Inventario.
- Pagos.
- Notificaciones.
- Usuarios y roles.
- Auditoría.
- Reportes operativos.
- Frontend administrativo en Angular.

Capacidades técnicas esperadas

- API HTTP documentada.
- Separación clara entre dominio, aplicación, infraestructura y presentación.
- Persistencia relacional con Entity Framework Core.
- Migraciones y configuración de entidades.
- Consultas eficientes con LINQ.
- DTOs, validaciones y mapeos.
- Frontend Angular desacoplado.
- Formularios reactivos.
- Manejo de estado básico.
- Pruebas unitarias backend y frontend.
- Contenedores Docker.
- Pipeline CI/CD.
- Buenas prácticas de seguridad.

Semana 1 - Fundamentos, arquitectura y diseño base

Día 1 - Fundamentos web, APIs y comunicación cliente-servidor

Concepto base: Una aplicación full stack funciona como una conversación entre cliente y servidor.

Angular consume recursos expuestos por una API, y ASP.NET Core procesa reglas de negocio, persistencia y respuestas.

Temas

- Cliente-servidor.
- HTTP y HTTPS.
- Métodos HTTP.
- Códigos de estado.
- API como contrato.
- REST.
- JSON.
- Tokens, autenticación y autorización.
- Diferencia entre API, controlador, servicio, recurso y endpoint.
- Comunicación Angular HTTP Client con backend.

Actividad práctica

- Diseñar el contrato inicial de una API para crear y consultar órdenes.
- Definir recursos, métodos, códigos de respuesta y errores esperados.
- Crear estructura inicial de solución .NET y aplicación Angular.

Preguntas de validación

- ¿Por qué una API es un contrato y no solo una URL?
- ¿Cuándo usarías 400, 401, 403, 404, 409 y 500?
- ¿Qué problema resuelve HTTPS frente a HTTP?
- ¿Qué responsabilidad tiene Angular y cuál tiene la API?

Día 2 - Clean Code, seguridad y fundamentos de ingeniería de software

Concepto base: El software debe ser mantenible, no solo funcional. El código limpio permite evolucionar el sistema sin romperlo.

Temas

- Clean Code.
- Principios SOLID.
- Bajo acoplamiento y alta cohesión.
- OWASP Top 10.
- SQL Injection.
- XSS.
- Validación de entrada.
- Manejo seguro de errores.
- Configuración segura.
- Secretos y variables de entorno.
- Buenas prácticas en C#.
- Buenas prácticas en TypeScript.

Actividad práctica

- Crear una lista de riesgos para la API de órdenes.
- Definir validaciones de entrada.
- Diseñar respuestas de error sin exponer detalles internos.
- Identificar posibles riesgos XSS en Angular.

Preguntas de validación

- ¿Por qué no se debe mostrar un stack trace al consumidor de una API?
- ¿Cuál es la diferencia entre validación, sanitización y autorización?
- ¿Cómo ayuda Entity Framework a prevenir SQL Injection?
- ¿Por qué Angular no debe confiar en datos provenientes del backend sin validación?

Día 3 - Clean Architecture / Arquitectura hexagonal en .NET

Concepto base: La arquitectura protege el negocio de los detalles técnicos. El dominio no debe depender de ASP.NET Core, Angular, Entity Framework ni la base de datos.

Temas

- Dominio.
- Casos de uso.
- Entidades.
- Value Objects.
- Puertos de entrada.
- Puertos de salida.
- Adaptadores primarios.
- Adaptadores secundarios.
- Inversión de dependencias.
- Separación entre reglas de negocio y detalles técnicos.
- Estructura recomendada en solución .NET.

src/

Orders.Domain

Orders.Application

Orders.Infrastructure

Orders.Api

Orders.Web

tests/

Orders.UnitTests

Orders.IntegrationTests

Actividad práctica

- Modelar el caso de uso CreateOrder.
- Definir entidades, objetos de valor y reglas de negocio.
- Crear interfaces de repositorio en la capa de aplicación.
- Separar claramente dominio, aplicación, infraestructura y API.

Preguntas de validación

- ¿Por qué el dominio no debe depender de Entity Framework?
- ¿Qué diferencia hay entre entidad de dominio y entidad de persistencia?
- ¿Dónde ubicarías una integración con un proveedor de pagos?
- ¿Dónde debe vivir la interfaz del repositorio?

Día 4 - Fundamentos de C# moderno y programación funcional aplicada

Concepto base: C# permite escribir código expresivo, seguro y mantenible usando objetos, funciones, LINQ, records, pattern matching e inmutabilidad.

Temas

- C# moderno.
- Clases, records y structs.
- Null safety.
- LINQ.
- Expresiones lambda.
- Delegados.

- Métodos de extensión.
- Inmutabilidad.
- Pattern matching.
- Result pattern.
- Excepciones vs errores funcionales.
- Programación funcional aplicada a reglas de negocio.

Actividad práctica

- Calcular totales de una orden usando funciones puras.
- Filtrar productos disponibles.
- Agrupar productos por categoría.
- Aplicar descuentos mediante funciones componibles.
- Crear objetos de valor para Money, Quantity y OrderId.

Preguntas de validación

- ¿Qué hace que una función sea pura?
- ¿Cuándo usarías record en C#?
- ¿Qué ventaja tiene LINQ frente a ciclos imperativos?
- ¿Cuándo una excepción no debería usarse para controlar flujo de negocio?

Día 5 - Entregable práctico 1

Entregable 1: API base de órdenes con Clean Architecture

Historia de usuario

Como usuario del sistema de compras, quiero crear una orden con productos, cantidades y datos del comprador, para iniciar el proceso de compra de forma controlada y validada.

Criterios de aceptación

- La solución debe exponer una API para crear órdenes.
- El dominio debe validar que una orden tenga al menos un producto.
- El dominio debe validar cantidades mayores a cero.
- El dominio debe calcular el total de la orden.
- El dominio no debe depender de ASP.NET Core ni Entity Framework.
- Debe existir un caso de uso CreateOrder.
- Debe existir un adaptador web.
- Debe existir un repositorio simulado en memoria.
- Debe existir documentación básica del contrato API.
- Deben existir pruebas unitarias del caso de uso principal.

Evidencias esperadas

- Diagrama simple de arquitectura.
- Código fuente organizado por capas.
- Pruebas unitarias ejecutables.

- Colección o contrato de API.
- README con decisiones técnicas.

Revisión técnica

El desarrollador debe explicar:

- Qué parte representa el dominio.
- Qué parte representa el caso de uso.
- Qué interfaces creó.
- Qué adaptadores implementó.
- Qué reglas de negocio probó.
- Qué decisiones tomó para evitar acoplamiento.

Semana 2 - Angular, Entity Framework, pruebas y documentación

Día 6 - Pruebas unitarias backend y frontend

Concepto base: Una prueba es un contrato ejecutable. Si la documentación dice cómo debería comportarse el sistema, la prueba lo verifica automáticamente.

Temas

- Pirámide de pruebas.
- Pruebas unitarias en .NET.
- xUnit, NUnit o MSTest.
- Moq, NSubstitute o FakeItEasy.
- Pruebas de servicios.
- Pruebas de controladores.
- Pruebas de componentes Angular.
- Jasmine/Karma o Jest.
- Testing Library.
- Cobertura útil vs cobertura cosmética.

Actividad práctica

- Probar validaciones del caso de uso CreateOrder.
- Probar errores de entrada.
- Probar cálculo del total.
- Simular el puerto de persistencia.
- Crear prueba simple de un componente Angular.

Preguntas de validación

- ¿Qué diferencia hay entre mock, stub y fake?
- ¿Qué pruebas pertenecen al dominio y cuáles a infraestructura?
- ¿Por qué una cobertura alta no garantiza calidad?
- ¿Qué debería probarse en un componente Angular?

Día 7 - Documentación de API y seguimiento semanal 1

Concepto base: Una API sin documentación es difícil de consumir. Swagger/OpenAPI permite que frontend, QA y otros equipos entiendan el contrato.

Temas

- OpenAPI/Swagger.
- Contratos de API.
- Versionamiento.
- DTOs.
- Errores estándar.
- Ejemplos de request/response.
- Criterios de aceptación verificables.
- Separación entre DTOs y entidades de dominio.

Actividad práctica

- Documentar la API de órdenes.
- Agregar ejemplos de respuestas exitosas y fallidas.
- Definir códigos de error funcionales.
- Crear servicio Angular para consumir la API.

Seguimiento semanal 1

Objetivo

Validar comprensión de fundamentos, seguridad, arquitectura, pruebas y API.

Preguntas sugeridas

- Explícame el flujo completo desde el componente Angular hasta el dominio.
- ¿Qué pasaría si mañana cambiamos la base de datos?
- ¿Qué reglas de negocio no deben estar en el controlador?
- ¿Cómo validaste los errores?
- ¿Qué vulnerabilidades básicas evitaste?

Señales de alerta

- El caso de uso depende directamente de ASP.NET Core.
- Las reglas de negocio están en el controlador.
- Las pruebas solo validan caminos felices.
- No existe manejo consistente de errores.
- Angular consume modelos internos del dominio directamente.
- La API no tiene contrato claro.

Día 8 - Entity Framework Core, SQL y diseño de datos

Concepto base: Entity Framework Core permite trabajar con persistencia relacional usando objetos, pero el modelo de dominio no debe quedar acoplado al ORM.

Temas

- Modelo relacional.
- DbContext.
- DbSet.
- Entidades EF.
- Fluent API.
- Data annotations.
- Migraciones.
- Relaciones uno a muchos.
- Relaciones muchos a muchos.
- Índices.
- Transacciones.
- Integridad referencial.
- Repositorios como adaptadores secundarios.
- Unit of Work.
- Tracking vs No Tracking.
- Consultas con LINQ.

Actividad práctica

- Diseñar persistencia para órdenes.
- Crear entidades de infraestructura para EF Core.
- Configurar relaciones.
- Crear migración inicial.
- Implementar repositorio con Entity Framework.
- Diseñar consulta de órdenes por cliente.

Preguntas de validación

- ¿Por qué no deberías exponer entidades EF directamente en la API?
- ¿Qué diferencia hay entre AsNoTracking y una consulta con tracking?
- ¿Qué problema resuelve una migración?
- ¿Qué riesgo tiene una consulta LINQ mal diseñada?
- ¿Cuándo usarías transacciones?

Día 9 - Angular: fundamentos, componentes, servicios y formularios

Concepto base: Angular organiza la interfaz en componentes, servicios y estructura por features. La UI debe estar separada de la lógica de negocio del backend.

Temas

- Componentes Angular.
- Standalone components.
- Templates.
- Data binding.
- Event binding.
- Servicios.

- HttpClient.
- Rutas.
- Guards.
- Interceptors.
- Formularios reactivos.
- Validaciones frontend.
- Manejo de errores HTTP.
- RxJS básico.
- Observables.

Actividad práctica

- Crear pantalla para registrar órdenes.
- Crear formulario reactivo.
- Consumir endpoint de creación de órdenes.
- Mostrar errores funcionales provenientes de la API.
- Crear pantalla para consultar órdenes por cliente.

Preguntas de validación

- ¿Cuál es la diferencia entre validación frontend y validación backend?
- ¿Por qué no se debe poner lógica crítica solo en Angular?
- ¿Qué problema resuelve un interceptor?
- ¿Cuándo usarías un guard?
- ¿Qué diferencia hay entre Promise y Observable?

Día 10 - Entregable práctico 2

Entregable 2: consultas, documentación, pruebas y persistencia con Entity Framework

Historia de usuario

Como asesor comercial, quiero consultar las órdenes de un cliente, para revisar su historial de compras y dar soporte oportuno.

Criterios de aceptación

- Debe existir endpoint para consultar órdenes por cliente.
- La consulta debe estar documentada en Swagger.
- Debe existir persistencia real usando Entity Framework Core.
- Deben existir migraciones.
- Deben existir pruebas unitarias del caso de uso.
- Deben existir pruebas del adaptador web.
- Debe existir manejo de errores para cliente inexistente o sin órdenes.
- Debe existir propuesta de paginación.
- Angular debe permitir consultar órdenes desde una pantalla.

- Angular debe mostrar estados de carga, éxito y error.

Evidencias esperadas

- Contrato OpenAPI actualizado.
- Migración inicial.
- DbContext configurado.
- Repositorio con Entity Framework.
- Pruebas automatizadas.
- Pantalla Angular funcional.
- README con decisiones de persistencia.
- Checklist de seguridad básica.

Revisión técnica

El desarrollador debe explicar:

- Cómo diseñó el modelo de datos.
- Qué reglas probó.
- Cómo evita acoplar dominio con Entity Framework.
- Cómo manejaría paginación.
- Qué datos no expondría en la API.
- Cómo Angular consume la API sin depender de detalles internos.

Semana 3 - Seguridad, autenticación, integración y comunicación asíncrona

Día 11 - ASP.NET Core avanzado: middleware, filtros y manejo de errores

Concepto base: Una API madura centraliza errores, validaciones, autenticación, logging y comportamiento transversal.

Temas

- Middleware.
- Filtros.
- Exception handling global.
- Problem Details.
- Validación con FluentValidation.
- DTOs de entrada y salida.
- Mapeo entre DTOs y dominio.
- AutoMapper o mapeo manual.
- CORS.
- Rate limiting.
- Health checks.
- Configuración por ambiente.

Actividad práctica

- Implementar manejo global de errores.
- Estandarizar respuestas con Problem Details.
- Agregar validaciones con FluentValidation.
- Configurar CORS para Angular.
- Crear health check básico.

Preguntas de validación

- ¿Por qué conviene centralizar el manejo de errores?
- ¿Qué diferencia hay entre error técnico y error funcional?
- ¿Por qué CORS no reemplaza autenticación?
- ¿Dónde ubicarías validaciones de formato y reglas de negocio?

Día 12 - Autenticación y autorización con JWT

Concepto base: La autenticación responde quién eres. La autorización define qué puedes hacer.

Temas

- Autenticación.
- Autorización.
- JWT.
- Claims.
- Roles.
- Políticas.
- Refresh tokens.
- Protección de endpoints.
- Guards en Angular.
- Interceptors para tokens.
- Almacenamiento seguro de tokens.
- Riesgos de XSS y robo de tokens.

Actividad práctica

- Crear login simulado.
- Proteger endpoints de órdenes.
- Agregar roles Admin, Seller y Viewer.
- Crear guard en Angular.
- Agregar interceptor para enviar token.
- Mostrar u ocultar opciones según rol.

Preguntas de validación

- ¿Qué diferencia hay entre autenticación y autorización?
- ¿Qué información no debería ir dentro de un JWT?
- ¿Por qué guardar tokens en localStorage puede ser riesgoso?
- ¿Cómo protegerías una ruta Angular?

Día 13 - Comunicación asíncrona y eventos de dominio

Concepto base: Los módulos no siempre deben llamarse directamente. Pueden publicar eventos para que otros módulos reaccionen sin acoplarse.

Temas

- Eventos de dominio.
- Eventos de integración.
- Publicador.
- Consumidor.
- Broker.
- Colas.
- Tópicos.
- Idempotencia.
- Reintentos.
- Dead letter queue.
- Consistencia eventual.
- Outbox pattern con Entity Framework.

Actividad práctica

- Definir evento OrderCreated.
- Definir consumidores potenciales: inventario, pagos y notificaciones.
- Diseñar estrategia de idempotencia.
- Diseñar tabla Outbox.
- Publicar evento de forma simulada.

Preguntas de validación

- ¿Qué diferencia hay entre evento de dominio y evento de integración?
- ¿Por qué un consumidor debe ser idempotente?
- ¿Qué problema resuelve una dead letter queue?
- ¿Por qué el patrón Outbox ayuda con consistencia?

Día 14 - Integración backend/frontend y seguimiento semanal 2

Concepto base: Una aplicación full stack debe manejar correctamente errores, estados de carga, seguridad, contratos y cambios de API.

Temas

- Contratos compartidos.
- DTOs frontend.
- Servicios Angular.
- Manejo de errores HTTP.
- Interceptors.
- Guards.
- Estados de carga.
- Paginación.

- Filtros.
- Ordenamiento.
- Evitar acoplamiento entre UI y backend.
- Mock de API para desarrollo frontend.

Actividad práctica

- Integrar pantalla de listado de órdenes.
- Agregar filtros por cliente y estado.
- Agregar paginación.
- Mostrar errores funcionales.
- Simular backend caído.
- Documentar contrato frontend/backend.

Seguimiento semanal 2

Objetivo

Validar comprensión de Angular, Entity Framework, seguridad, integración y eventos.

Preguntas sugeridas

- Explícame el flujo desde Angular hasta Entity Framework.
- ¿Dónde podrían aparecer problemas de seguridad?
- ¿Cómo evitarías que Angular dependa del modelo de base de datos?
- ¿Qué evento publicas al crear una orden?
- ¿Cómo evitarías procesar dos veces el mismo evento?

Señales de alerta

- Entidades EF expuestas directamente en la API.
- Angular usa modelos internos del backend sin DTOs.
- No existe manejo de errores centralizado.
- Tokens manejados sin criterio de seguridad.
- Eventos con demasiada información interna.
- No existe estrategia de idempotencia.

Día 15 - Entregable práctico 3

Entregable 3: aplicación full stack con seguridad, EF Core y eventos

Historia de usuario

Como sistema de ventas, quiero crear órdenes desde una interfaz web segura, persistirlas correctamente y publicar un evento cuando sean creadas, para que otros módulos puedan reaccionar sin acoplarse directamente.

Criterios de aceptación

- Angular debe permitir crear y consultar órdenes.
- La API debe proteger endpoints mediante JWT.

- Debe existir autorización por roles.
- La creación de orden debe persistir usando EF Core.
- La creación de orden debe generar evento OrderCreated.
- El evento debe tener identificador único.
- El evento debe tener fecha de ocurrencia.
- El evento debe contener solo información necesaria.
- Debe existir adaptador para publicación de eventos, aunque sea simulado.
- Deben existir pruebas backend.
- Deben existir pruebas frontend básicas.
- Debe existir documentación de seguridad y eventos.

Evidencias esperadas

- Pantallas Angular funcionales.
- API protegida.
- Migraciones EF Core.
- Contrato OpenAPI.
- Contrato del evento.
- Pruebas automatizadas.
- README con decisiones de seguridad.
- Evidencia de ejecución local.

Revisión técnica

El desarrollador debe explicar:

- Cómo protegió la API.
- Cómo Angular maneja autenticación.
- Cómo desacopló el dominio de Entity Framework.
- Cómo desacopló el caso de uso del broker.
- Cómo probaría el flujo sin infraestructura real.
- Cómo manejaría eventos duplicados.

Semana 4 - DevOps, Docker, Kubernetes, nube y observabilidad

Día 16 - Consultas avanzadas, performance y optimización con Entity Framework

Concepto base: Entity Framework simplifica el acceso a datos, pero un mal uso puede generar consultas lentas, exceso de memoria o problemas de concurrencia.

Temas

- LINQ avanzado.
- Includes.
- Proyecciones.
- Consultas paginadas.
- AsNoTracking.
- Lazy loading vs eager loading.

- N+1 queries.
- Índices.
- Transacciones.
- Concurrencia optimista.
- RowVersion.
- Bulk operations.
- Separación lectura/escritura.
- CQRS básico.

Actividad práctica

- Optimizar consulta de órdenes.
- Evitar N+1 queries.
- Crear proyección hacia DTO.
- Agregar paginación real.
- Agregar índice a campos de búsqueda.
- Documentar decisiones de performance.

Preguntas de validación

- ¿Qué es el problema N+1?
- ¿Por qué conviene proyectar a DTO en consultas?
- ¿Cuándo usarías AsNoTracking?
- ¿Cómo manejarías concurrencia en inventario?
- ¿Qué costo tiene agregar índices?

Día 17 - Resiliencia, consistencia y patrones de integración

Concepto base: En sistemas distribuidos, fallar es normal. Una arquitectura madura contiene, detecta y se recupera de los fallos.

Temas

- Timeout.
- Retry.
- Circuit breaker.
- Bulkhead.
- Rate limit.
- Outbox pattern.
- Inbox pattern.
- Saga.
- Idempotency key.
- Consistencia eventual.
- Trazabilidad distribuida.
- Correlation id.
- Polly en .NET.

Actividad práctica

- Diseñar flujo de orden, inventario y pago usando eventos.
- Definir qué pasa si pago falla.
- Definir compensación.
- Definir trazabilidad con correlation id.
- Diseñar estrategia de reintentos.

Preguntas de validación

- ¿Por qué no siempre conviene usar transacciones distribuidas?
- ¿Qué problema resuelve el patrón Outbox?
- ¿Qué diferencia hay entre retry y circuit breaker?
- ¿Cómo evitarías duplicar pagos?

Día 18 - Angular avanzado: arquitectura frontend y mantenibilidad

Concepto base: Una aplicación Angular debe organizarse por dominios, no solo por tipos técnicos. La UI también requiere arquitectura.

Temas

- Arquitectura por features.
- Lazy loading.
- Guards.
- Interceptors.
- Manejo de estado.
- Servicios de aplicación.
- Componentes presentacionales y contenedores.
- Reactive Forms avanzado.
- Validadores personalizados.
- RxJS operators.
- Manejo de suscripciones.
- Signals.
- Manejo de errores globales.
- Accesibilidad básica.

Actividad práctica

- Reorganizar Angular por módulos o features.
- Crear feature orders.
- Separar componentes contenedores y presentacionales.
- Implementar filtros reactivos.
- Manejar errores globales desde interceptor.
- Agregar validadores personalizados.

Preguntas de validación

- ¿Por qué separar componente contenedor y presentacional?
- ¿Qué problema resuelve lazy loading?

- ¿Cuándo usarías un servicio de estado?
- ¿Qué riesgo tiene una suscripción no liberada?
- ¿Qué validaciones deben vivir en frontend y cuáles en backend?

Día 19 - Observabilidad, logs, métricas y trazas

Concepto base: Operar sin observabilidad es conducir de noche sin luces. Logs, métricas y trazas permiten saber qué ocurrió, dónde y por qué.

Temas

- Logs estructurados.
- Serilog.
- Correlation id.
- Métricas técnicas.
- Métricas de negocio.
- Trazas distribuidas.
- Health checks.
- Readiness y liveness.
- Alertas.
- Monitoreo de errores frontend.
- Logging seguro.
- Datos sensibles en logs.

Actividad práctica

- Agregar correlation id al flujo de orden.
- Definir logs relevantes.
- Diseñar métricas de negocio.
- Crear health checks.
- Documentar qué datos no deben registrarse.
- Agregar manejo global de errores en Angular.

Preguntas de validación

- ¿Qué diferencia hay entre log, métrica y traza?
- ¿Qué datos no deberían aparecer en logs?
- ¿Por qué readiness y liveness no son lo mismo?
- ¿Cómo rastrearías una orden desde Angular hasta base de datos?

Día 20 - Entregable práctico 4

Entregable 4: flujo full stack resiliente y observable

Historia de usuario

Como sistema de órdenes, quiero coordinar inventario, pago y notificación mediante eventos, mostrando trazabilidad desde Angular hasta backend, para procesar compras de forma desacoplada y resiliente.

Criterios de aceptación

- Debe existir diseño de flujo entre órdenes, inventario, pagos y notificaciones.
- Debe existir al menos un productor y un consumidor de eventos.
- Debe existir manejo de duplicados.
- Debe existir estrategia de reintentos.
- Debe existir definición de DLQ o mecanismo equivalente.
- Debe existir correlation id en el flujo.
- Debe existir documentación de eventos.
- Deben existir pruebas del consumidor.
- Angular debe mostrar estado de la orden.
- La API debe exponer health checks.
- Debe existir logging estructurado.
- Debe existir comparación documentada entre comunicación HTTP y eventos.

Evidencias esperadas

- Diagrama de eventos.
- Contrato de eventos.
- Código del consumidor.
- Pruebas automatizadas.
- README de resiliencia.
- Evidencia de logs con correlation id.
- Pantalla Angular mostrando estado de la orden.

Revisión técnica

El desarrollador debe explicar:

- Por qué eligió eventos.
- Cómo garantiza idempotencia.
- Cómo diagnosticaría un mensaje fallido.
- Cómo reprocesaría eventos.
- Cómo protegería información sensible en mensajes y logs.
- Cómo Angular informa estados sin acoplarse al proceso interno.

Semana 5 - Docker, CI/CD, Kubernetes, AWS y cierre

Día 21 - Docker, Git y CI/CD con Azure DevOps

Concepto base: Docker empaqueta la aplicación con sus dependencias. CI/CD automatiza compilación, pruebas, inspección y despliegue.

Temas

- Git flow o trunk-based development.
- Pull requests.
- Revisión de código.
- Dockerfile para .NET.

- Dockerfile para Angular.
- Docker Compose.
- Variables de entorno.
- Gestión de secretos.
- Pipeline YAML.
- Stages: build, test, quality, package, deploy.
- Publicación de artefactos o imágenes.
- Quality gates.

Actividad práctica

- Crear Dockerfile para API .NET.
- Crear Dockerfile para Angular.
- Crear Docker Compose con API, frontend y base de datos.
- Crear pipeline de CI.
- Ejecutar pruebas en pipeline.
- Publicar imagen o artefacto.

Seguimiento semanal 3

Objetivo

Validar resiliencia, Angular avanzado, Entity Framework, observabilidad y DevOps inicial.

Preguntas sugeridas

- ¿Qué diferencia hay entre imagen y contenedor?
- ¿Qué etapas debe tener un pipeline mínimo de calidad?
- ¿Qué variables no deben quedar dentro de la imagen?
- ¿Cómo revisas si un PR está listo para aprobarse?
- ¿Qué harías si el consumidor procesa dos veces un evento?
- ¿Cómo validarías que Angular compila correctamente en pipeline?

Señales de alerta

- Dockerfile con secretos.
- Imagen demasiado pesada.
- Pipeline que despliega sin pruebas.
- Falta de trazabilidad en eventos.
- No hay separación de configuración por ambiente.
- Angular y API tienen configuración quemada en código.

Día 22 - Kubernetes: fundamentos operativos

Concepto base: Kubernetes administra contenedores. Decide dónde corre cada contenedor, lo reinicia si falla, escala réplicas y expone servicios de forma controlada.

Temas

- Cluster.
- Node.

- Pod.
- Deployment.
- Service.
- ConfigMap.
- Secret.
- Ingress.
- Requests y limits.
- Liveness probe.
- Readiness probe.
- Horizontal scaling.
- Namespaces.
- Rolling update.

Actividad práctica

- Crear manifiestos para desplegar API.
- Crear manifiestos para desplegar Angular.
- Definir variables por ambiente.
- Configurar probes.
- Definir recursos mínimos.
- Exponer frontend y API mediante servicios.

Preguntas de validación

- ¿Por qué no se debe desplegar directamente un Pod manual?
- ¿Qué diferencia hay entre ConfigMap y Secret?
- ¿Qué pasa si readiness falla?
- ¿Por qué definir requests y limits?
- ¿Cómo escalarías API y frontend de forma independiente?

Día 23 - AWS: fundamentos para aplicaciones full stack

Concepto base: AWS provee piezas administradas para construir sistemas escalables. La clave no es usar muchos servicios, sino elegir los necesarios para seguridad, costo, resiliencia y operación.

Temas

- Regiones y zonas de disponibilidad.
- IAM.
- VPC.
- Subnets públicas y privadas.
- Load Balancer.
- ECS/EKS.
- RDS.
- S3.
- CloudFront.
- SQS.
- CloudWatch.

- Secrets Manager.
- Principio de menor privilegio.
- Despliegue de SPA Angular.
- API backend privada o pública controlada.

Actividad práctica

- Diseñar arquitectura cloud para API .NET y Angular.
- Separar componentes públicos y privados.
- Definir base de datos en subred privada.
- Definir gestión de secretos.
- Definir monitoreo básico.
- Diseñar estrategia de despliegue de Angular en S3 + CloudFront o contenedor.

Preguntas de validación

- ¿Por qué una base de datos no debe estar en una subred pública?
- ¿Qué significa menor privilegio?
- ¿Cuándo usarías SQS frente a comunicación HTTP directa?
- ¿Dónde desplegarías Angular y por qué?
- ¿Qué servicio usarías para secretos?

Día 24 - Seguridad aplicada, calidad y preparación de entrega

Concepto base: La calidad no es una fase; es una propiedad transversal. Cada commit debe acercar el sistema a producción.

Temas

- Seguridad en APIs.
- Seguridad en Angular.
- Validación centralizada.
- Manejo de secretos.
- Dependencias vulnerables.
- Análisis estático.
- Quality gates.
- Revisión de PR.
- Pruebas mínimas antes de merge.
- Buenas prácticas de documentación técnica.
- Headers de seguridad.
- CORS correcto.
- Protección contra XSS.
- Protección contra CSRF según estrategia de autenticación.

Actividad práctica

- Ejecutar checklist de seguridad del proyecto.
- Revisar dependencias backend.
- Revisar dependencias frontend.

- Revisar logs sensibles.
- Validar configuración de CORS.
- Preparar entrega del quinto incremento.

Preguntas de validación

- ¿Qué controles mínimos deberían estar en un pipeline?
- ¿Qué información no debe registrarse en logs?
- ¿Cómo validarías que una dependencia no tiene vulnerabilidades conocidas?
- ¿Por qué CORS abierto es riesgoso?
- ¿Qué datos nunca deberían exponerse al frontend?

Día 25 - Entregable práctico 5

Entregable 5: aplicación full stack contenerizada con pipeline y manifiestos Kubernetes

Historia de usuario

Como equipo DevOps, quiero construir, probar, contenerizar y desplegar la API .NET y la aplicación Angular, para tener un flujo repetible y confiable hacia ambientes superiores.

Criterios de aceptación

- Debe existir Dockerfile optimizado para API .NET.
- Debe existir Dockerfile optimizado para Angular.
- Debe existir configuración para ejecución local multi-servicio.
- Debe existir base de datos configurada en Docker Compose.
- Debe existir pipeline con build, test y análisis básico de calidad.
- Las pruebas deben ejecutarse automáticamente.
- Deben existir manifiestos Kubernetes.
- Deben existir ConfigMaps y Secrets conceptualizados correctamente.
- Deben existir probes de salud.
- Deben existir requests y limits.
- Debe existir documentación para ejecutar localmente y desplegar.

Evidencias esperadas

- Dockerfile de API.
- Dockerfile de Angular.
- Archivo de composición local.
- Pipeline YAML.
- Manifiestos Kubernetes.
- README operativo.
- Evidencia de ejecución de pruebas en pipeline.
- Evidencia de ejecución local.

Revisión técnica

El desarrollador debe explicar:

- Cómo optimizó las imágenes.
- Cómo separó configuración de código.
- Qué etapas tiene el pipeline.
- Cómo fallaría el pipeline ante baja calidad.
- Cómo Kubernetes sabe si la aplicación está lista.
- Cómo desplegaría Angular y API de forma independiente.

Día 26 - Diseño de arquitectura final

Concepto base: Una arquitectura se evalúa por sus trade-offs. No existe una única solución perfecta; existe una solución adecuada al contexto, restricciones y objetivos.

Temas

- Diagramas C4.
- Diagrama de contexto.
- Diagrama de contenedores.
- Diagrama de componentes.
- Decisiones arquitectónicas.
- ADRs.
- Trade-offs.
- Costos operativos.
- Escalabilidad.
- Seguridad.
- Resiliencia.
- Mantenibilidad.
- Evolución del sistema.

Actividad práctica

- Crear diagrama de contexto.
- Crear diagrama de contenedores.
- Crear diagrama de componentes de la API de órdenes.
- Crear diagrama de componentes Angular.
- Documentar decisiones arquitectónicas.
- Documentar trade-offs.

Preguntas de validación

- ¿Qué sacrificaste al elegir eventos?
- ¿Qué parte escalarías primero y por qué?
- ¿Qué decisión cambiarías si el volumen creciera 10 veces?
- ¿Qué decisión cambiarías si el equipo frontend creciera?
- ¿Qué decisión cambiarías si la base de datos tuviera alta concurrencia?

Día 27 - Integración end-to-end y revisión de deuda técnica

Concepto base: Integrar no es juntar piezas; es verificar que trabajan como un sistema coherente. La deuda técnica aceptable es consciente, visible y gestionada.

Temas

- Integración end-to-end.
- Revisión de deuda técnica.
- Refactoring seguro.
- Pruebas de regresión.
- Contratos entre frontend y backend.
- Checklist de readiness para producción.
- Gestión de issues técnicos.
- Priorización de deuda.

Actividad práctica

- Ejecutar revisión completa del proyecto.
- Identificar deuda técnica.
- Clasificar deuda: crítica, importante, menor.
- Crear plan de mejora.
- Ejecutar prueba end-to-end manual o automatizada.
- Validar flujo completo desde Angular hasta base de datos.

Preguntas de validación

- ¿Qué deuda técnica aceptarías temporalmente?
- ¿Qué deuda no permitirías pasar a producción?
- ¿Cómo evitarías romper contratos existentes?
- ¿Cómo validarías que Angular y backend siguen siendo compatibles?

Día 28 - Seguimiento semanal 4 y simulación de revisión técnica

Seguimiento semanal 4

Objetivo

Validar dominio integral del diseño, implementación, despliegue, seguridad, operación y experiencia full stack.

Dinámica sugerida

El desarrollador debe presentar el sistema como si estuviera en una revisión técnica real ante arquitectura, desarrollo, QA y DevOps.

Preguntas sugeridas

- Explícame la arquitectura completa en 5 minutos.
- ¿Dónde están las reglas de negocio?
- ¿Cómo pruebas el flujo crítico?

- ¿Qué ocurre si el broker falla?
- ¿Qué ocurre si la base de datos responde lento?
- ¿Cómo escalarías consumidores?
- ¿Cómo escalarías Angular?
- ¿Cómo evitarías duplicados?
- ¿Cómo se despliega la API?
- ¿Cómo se despliega Angular?
- ¿Cómo se monitorea?
- ¿Qué vulnerabilidades mitigaste?
- ¿Cómo usaste Entity Framework sin acoplar el dominio?

Señales de alerta

- No puede explicar decisiones arquitectónicas.
- Confunde dominio con infraestructura.
- Confunde entidad de dominio con entidad EF.
- No sabe diagnosticar fallos distribuidos.
- No puede justificar seguridad o despliegue.
- Angular contiene lógica crítica de negocio.
- No existe estrategia clara de migraciones.

Día 29 - Preparación del entregable final

Concepto base: Una entrega profesional no solo contiene código; contiene contexto, evidencia, decisiones, pruebas, riesgos y guía de operación.

Actividad práctica

- Completar README final.
- Completar diagramas.
- Completar pruebas faltantes.
- Completar documentación de API.
- Completar documentación de eventos.
- Completar documentación frontend.
- Completar pipeline.
- Completar manifiestos.
- Preparar demo técnica.
- Preparar checklist de seguridad.
- Preparar lista de deuda técnica.

Checklist previo

- El backend compila.
- El frontend compila.
- Las pruebas backend pasan.
- Las pruebas frontend pasan.
- La API está documentada.
- Los eventos están documentados.

- Entity Framework tiene migraciones.
- El flujo crítico funciona.
- Hay Dockerfile para API.
- Hay Dockerfile para Angular.
- Hay pipeline.
- Hay manifiestos Kubernetes.
- Hay checklist de seguridad.
- Hay decisiones arquitectónicas documentadas.
- Hay evidencia de ejecución local.

Día 30 - Entregable práctico 6 y evaluación final

Entregable 6: solución full stack lista para revisión arquitectónica

Historia de usuario épica

Como compañía, quiero contar con una plataforma de órdenes moderna, segura, documentada, desplegable y resiliente, para soportar procesos de compra escalables y mantenibles.

Criterios de aceptación

- Clean Architecture aplicada correctamente.
- Dominio independiente del framework.
- Dominio independiente de Entity Framework.
- API documentada.
- Aplicación Angular funcional.
- Formularios reactivos implementados.
- Seguridad básica aplicada.
- Autenticación y autorización implementadas.
- Persistencia con Entity Framework Core.
- Migraciones configuradas.
- Eventos documentados.
- Pruebas unitarias backend.
- Pruebas frontend básicas.
- Manejo de errores funcionales y técnicos.
- Integración con broker o simulación realista.
- Dockerfile y ejecución local.
- Pipeline CI/CD.
- Manifiestos Kubernetes.
- Diseño cloud en AWS.
- Observabilidad mínima definida.
- README técnico y operativo.

Evidencias esperadas

- Demo funcional.
- Repositorio organizado.

- Documentación técnica.
- Diagramas de arquitectura.
- Reporte de pruebas.
- Evidencia de pipeline.
- Evidencia de despliegue local o en clúster.
- Checklist de seguridad.
- Lista de deuda técnica.
- Documentación de migraciones.
- Documentación de variables de entorno.

Presentación final

El desarrollador debe presentar:

1. Problema funcional.
2. Arquitectura propuesta.
3. Flujo de creación de orden.
4. Diseño de dominio.
5. Diseño de persistencia con Entity Framework.
6. API y contratos.
7. Angular y experiencia de usuario.
8. Pruebas.
9. Eventos.
10. Resiliencia.
11. Seguridad.
12. Despliegue.
13. Riesgos y mejoras futuras.

Resumen de ejercicios prácticos cada 5 días

Día	Entregable	Enfoque principal	Resultado esperado
5	API base de órdenes	Clean Architecture, dominio, API, pruebas	Primer incremento funcional backend
10	Consultas y datos	Entity Framework, documentación, pruebas, Angular básico	API consultable, persistente y documentada
15	Full stack seguro	Angular, JWT, EF Core, eventos	Flujo full stack desacoplado y seguro
20	Flujo resiliente	Eventos, observabilidad, resiliencia, frontend de estado	Flujo preparado para fallos
25	Docker, CI/CD y Kubernetes	DevOps, contenedores, despliegue	Aplicación ejecutable y desplegable
30	Solución integral	Arquitectura, seguridad, nube, operación	Demo final de nivel avanzado